Introduction to Computer Science M – Spring 2025 - Assignment A2

version 1.0

Rules:

- **Submission Deadline: May 23 at 23:59**. Late submissions will incur the following penalties:
  - **May 24:** 10-point deduction
  - **May 25:** 20-point deduction
  - **May 26:** 30-point deduction

  Submissions will not be accepted after May 26.
- The assignment should be completed in groups already assigned in Moodle.
- Students will be randomly selected to provide an oral explanation of their solution to the exercises.
- Questions regarding the exercise statements are welcome during office hours. However, professor will not assess your solution.
- There is no guarantee that questions submitted via email will receive a response.
- The code must be implemented in C. Submitted code should compile (**no compilation errors** if compiled with gcc). This is the minimum requirement. Note that graders and professors will assess the correctness and quality of the submitted code. **Compilation warnings** will be translated to deduction in the grades.
- **Important:** Your programs should only use C features covered in weeks 1-8 of the course. In addition, the following restrictions apply:
  - Do not use `break` or `continue` statements.
  - The program must contain only one `return` statement.

  If a solution violates the above restriction, **it will receive a grade of 0**.
- You should submit a separate file containing the source code for each exercise (ensure the file names match the requirements for each task). For exercises 5 and 8, you can submit a .pdf file with your solution. Alternatively, you may submit a single zip file containing the code for all exercises.

Plagiarism and Academic Misconduct:

In cases of copying or plagiarism, exams will be placed under review. If plagiarism is confirmed, the case will be reported to the GTIIT Discipline Committee. **This includes submitting solutions found online or generated by AI tools.** All group members will be held accountable for any submission that violates this policy.

Exercise 1: Removing Redundant Blank Spaces [10 points]

Compose a C program `clean.c` utilizing `getchar()` and `putchar` to accept input and eliminate redundant blank spaces. Redundant blank spaces denote consecutive instances of space characters `' '` within the input. The program is designed to output the input with any redundant blank spaces removed.

For example, if the user inputs:

```
Hello    world!   This  is   a   test.
```

The program should output:

```
Hello world! This is a test.
```

Your program should preserve single spaces between words and remove the blank empty spaces at the beginning of line. Lines should be preserved.

If you run your program on the file `fullofblanks.txt` provided on Moodle, you should obtain the output contained in `presentation.txt` .

Exercise 2: Caesar Cipher Encoding [10 points]

In a Caesar cipher, every letter in a message is shifted by a consistent number of positions within the alphabet. For instance, with a shift value of 3, 'A' would transform into 'D', 'B' into 'E', and so forth. This shift operates circularly, meaning 'Z' would wrap around to 'C' with a shift of 3.

Within the file `ciphered.txt` , you'll discover a poem ciphered with a shift value of 5.

Develop a program `decode.c` capable of recovering the original poem. Your program should handle both uppercase and lowercase characters. Non-alphabetic characters, like punctuation marks or numbers, should remain unaltered in the decoded sequence.

Exercise 3: Random Password Generator Enhancement [15 points]

Consider the random password generator program introduced in Lecture 07. Give a modified version, called `generator.c` , to accept input from standard input for the following parameters:

- **Password Length:** The length of each password to be generated.
- **Number of Passwords:** The quantity of passwords to generate.
- **Biases for Categories:** The biases used for generating characters in different categories.

For instance, if the user inputs the sequence "100000 16 4 3 2 1", the program will generate `100,000` passwords of `16` characters each. Moreover, the biases are such that the chances of randomly generating a lowercase letter are 4 out of 10, an uppercase letter is 3 out of 10, a digit is 2 out of 10, and a symbol is 1 out of 10. Enhance the program according to the specifications above.

Your solution must not use arrays. If arrays are used, the grade will be 0.

Exercise 4: Quality of generated passwords [15 points]

Suppose we have the following classification of passwords:

- **Very Strong:** A password is categorized as very strong if the quantity of characters in each category pairwise differs by at most one. For example:
  - `P$ssA\12!` (there are two uppercase letters, 2 lowercase letters, 2 digits and 3 symbols). Contrastingly, `P@6ssW89rd!` is not very strong because it has 4 lowercase letters and two symbols.
- **Strong:** A password is classified as strong if it is not very strong but if it includes symbols from all categories. For example:
  - `P%ssW0rd!!`

- **Weak:** A weak password is not a strong password but contains uppercase and lowercase letters and digits (it does not include any special symbols). For example:
  - `Password123`
- **Very Weak:** if it is not weak.
  - `aabb123456` (composed by lowercase letters and digits)

Write a C program `quality.c` that reads a sequence of passwords from standard input (each terminated by `\n`) and determines the percentage of passwords in the sequence that fall into each category.

Each password must be read character by character using `getchar`, and arrays should not be used. If arrays are used, the grade will be 0. .

Exercise 5: Quality of Generated Passwords [5 points]

Describe how the Monte Carlo approach can be utilized alongside previous programs to assess the probability of generating a very strong password when considering the biases of 2, 3, 2, and 3 out of 10 for lowercase letters, uppercase letters, digits, and symbols respectively.

Exercise 6: Percentage of Passwords Starting with a Letter [10 points]

Create a program `starting.c` that, in conjunction with the password generator from exercise 1, utilizes the Monte Carlo approach to assess the chances of generating a password that does not start with a letter (either lowercase or uppercase) when considering the biases of `4`, `3`, `2`, and `1` out of 10 for lowercase letters, uppercase letters, digits, and symbols respectively.

Exercise 7: Pig Game [10 points]

Pig is a simple dice game where players take turns rolling a single die. Players can roll repeatedly, adding each result to their **turn total**, but lose their turn's points if they roll a **1**.

Rules

- **Players:** 2
- **Winning Condition:** First to reach **100+ points** wins.

Turn Mechanics

1. **Roll:**
   - Roll a 6-sided die.
   - If the roll is **1**:
     - Turn ends.
     - No points earned this turn.
   - If the roll is **2–6**:
     - Add the value to your **turn total**.
     - Choose to **roll again** or **hold**.
2. **Hold:**
   - Bank your turn total into your **score**.
   - Pass the die to the next player.

Write a C program `pig.c` that:

1. Randomly selects the starting player.
2. Alternates turns between two players.
3. For each turn:
    - Asks the user to **roll (r)** or **hold (h)**.
    - If **roll**:
        - Generates a random die roll (1–6).
        - On **1**: Ends turn, awards **0 points**.
        - On **2–6**: Adds to turn total, informs the user the accummulated points and repeats choice.
    - If **hold**: Adds turn total to player's score, switches turns.
4. Checks for a winner after each hold.
5. Announces the winning player and terminates.

Example Output

```
Player 1's turn (Score: 0)
Roll (r) or Hold (h)? r
You rolled: 5 | Turn total: 5
Roll (r) or Hold (h)? r
You rolled: 2 | Turn total: 7
Roll (r) or Hold (h)? h
Player 1's score: 7

Player 2's turn (Score: 0)
...
Player 1 wins with 103 points!
```

Exercise 8: One player vs computer [10 points]

Write `pigkeeppace.c` that modifies your Pig Game to support **1 human player vs a computer opponent** using the *"End race or keep pace"* strategy. The computer follows this logic each turn: If computer's total score is greater than 70 then always roll to win immediately. Otherwise, hold at `21 + (human_score − computer_score) / 8`. For instance, If human = 60, computer = 50, then the computer holds at `21 + (10/8) ≈ 22`.

Exercise 9: Which strategy is better[15 points]

Another common strategy is "Hold at 25", where a player continues rolling until accumulating 25 or more points in a single turn before holding. Modify your Pig game code into `autonomous.c` to make two computer players against each other: one using the "End race or keep pace" strategy and the other using "Hold at 25".

Use the Monte Carlo method to simulate multiple games between these strategies and determine which performs better.